Equipo 16	
Nombre: Lesly Adamaris Cotlame Nieto Christopher Bethancourt Contreras Francisco Mora Ruiz	**Matrícula**: 2226958 2226948 2226912
Nombre del curso: Componentes de lógica de negocios.	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: Diseño Arquitectural del Sistema
Fecha: 13/10/2026	
Bibliografia (Formato APA) • Beck, K. (2003). *Test-driven development: By example*. Addison-Wesley. • Cohn, M. (2004). *User stories applied: For agile software development*. Addison-Wesley. • Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley. • Fowler, M. (2003). *Patterns of enterprise application architecture*. Addison-Wesley. • Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall. • Millett, S., & Tune, N. (2015). *Patterns, principles, and practices of domain-driven design*. Wrox.	

Objetivo:

En esta actividad convertirán el análisis de negocio elaborado en la Unidad 2 en una propuesta de arquitectura de software. Trabajarán con la misma empresa y el mismo caso. La inteligencia artificial podrá apoyar el diseño y la revisión, pero cada decisión deberá justificarse con evidencia del proyecto

Procedimiento:

REPORTE RESUMEN DE LA ACTIVIDAD

Unidad 3 · Diseño Arquitectural del Sistema · Eventos Nova

Introducción

En la Unidad 3 seguimos trabajando con Eventos Nova, la misma empresa de la Unidad 2. No empezamos un caso nuevo: usamos el BRD, las User Stories y el glosario que ya teníamos para convertir las necesidades y las reglas del negocio en una propuesta de arquitectura de software. Antes de desarrollar un sistema completo queríamos ordenar las responsabilidades y comprobar, con evidencia, que el diseño representa lo que el negocio necesita.

Trabajamos las nueve etapas de la guía: preparar el caso, comparar patrones de lógica de negocio, hacer el mapa de dominio, diseñar la Service Layer, separar persistencia y transacciones, aplicar CQRS, revisar SOLID, documentar la arquitectura con UML y ADR, y validar con pruebas. Usamos Claude (Anthropic) para generar alternativas y revisar materiales, pero siempre comparamos sus respuestas con nuestros documentos y registramos en la bitácora las decisiones que tomamos como equipo.

El resultado es un prototipo de arquitectura en Python, con diagramas fuente y exportados, cuatro ADR, una matriz de trazabilidad, la documentación de la Unidad 2 y pruebas automatizadas. Todavía no es un sistema de producción: los repositorios y el Outbox son en memoria y los colaboradores externos están simulados.

1. Preparación del caso y casos de uso elegidos

1.1 Cómo funciona Eventos Nova

Eventos Nova organiza eventos como bodas, graduaciones y eventos empresariales. El proceso empieza con la información del Cliente y del Evento; el Coordinador prepara una propuesta con servicios, proveedores, costos y condiciones. Cuando el Cliente acepta la versión vigente se formaliza un contrato. Después se registran y verifican los pagos, se coordinan los proveedores y se atienden solicitudes posteriores (cambios, servicios adicionales o sustituciones). Al final se registra la cancelación o la finalización, según lo que realmente ocurrió.

En el BRD vimos que buena parte de la comunicación actual ocurre por WhatsApp, llamadas y correo. Eso puede producir versiones distintas de la información, dificultar el seguimiento de pagos y hacer que los proveedores no reciban un cambio a tiempo. Por eso nos concentramos en conservar una configuración vigente, registrar las decisiones y evitar confirmar una operación cuando faltan condiciones requeridas. La Tabla 1 resume las reglas que más pesaron en el diseño.

Tabla 1. Reglas de negocio que condicionaron el diseño

Área	Regla aplicada en el diseño
Propuesta	Se conserva el historial de versiones y solo una puede estar vigente. El Cliente acepta la vigente o solicita modificaciones antes de aceptarla.
Contratación y anticipo	Se requiere contrato firmado, política de no reembolso incluida y comunicada, y anticipo del 30 % registrado y verificado. El anticipo se paga dentro de 3 días hábiles posteriores a la firma; si no se recibe, el Coordinador contacta al Cliente, que dispone de 24 horas naturales desde ese contacto para pagar (el registro y la verificación pueden ser posteriores).
Saldo del contrato original	Debe liquidarse como máximo 3 días hábiles antes del evento. Si se activa la regularización, el Cliente tiene 24 horas naturales para pagar; al vencer, Administración revisa si pagó dentro de ese plazo (el registro y la verificación pueden ser posteriores) y, si no pagó, informa al Coordinador, quien registra la cancelación. No hay cancelación automática por el solo vencimiento; el Coordinador la registra en cuanto recibe el aviso y antes del inicio del evento.
Cambios	Se solicitan después de la contratación y antes del inicio del evento. En todos los casos la viabilidad se revisa con los proveedores involucrados. Con 7 días naturales o más no corresponde cargo tardío; con menos puede aplicar un cargo fijo de Eventos Nova. El importe de ese cargo no está fijado en los archivos revisados.
Servicio adicional	Debe solicitarse con al menos 5 días naturales; con menos no procede. Se considera el costo del proveedor, no un cargo adicional de Eventos Nova. Requiere disponibilidad, aprobación del Cliente, autorización de Administración, anexo y pago registrado/verificado cuando corresponda.
Pago de cambio o servicio adicional	Cuando la solicitud requiere pago, el Cliente tiene 24 horas naturales desde que se comunica el requerimiento para efectuarlo; el registro y la verificación de Administración pueden ser posteriores. Si se comprueba que no pagó dentro de ese plazo, la solicitud se cierra conforme a sus reglas y no se confirma el servicio o cambio.
Cancelación y finalización	El Cliente puede solicitar cancelar después de confirmada la contratación y antes del inicio; si el evento ya comenzó, la cancelación se rechaza. “Cancelado” y “Finalizado” son estados distintos. Al cancelarse el evento, las solicitudes en trámite se cierran con el motivo «Evento cancelado».
Sustitución de proveedor	El proveedor alternativo no se considera confirmado hasta cumplir las condiciones y aprobaciones que correspondan. La última configuración confirmada del Evento se conserva mientras el reemplazo esté pendiente.

Al revisar la arquitectura cuidamos especialmente dos cosas: no confundir el cierre sin contratación por anticipo no recibido con una cancelación del Evento, y no mezclar el cargo fijo de Eventos Nova con el costo del proveedor.

1.2 Continuidad con la Unidad 2

Usamos los documentos de la Unidad 2 como evidencia de entrada y como lenguaje común. Además, contamos con el BPMN de Eventos Nova, elaborado a partir de esos documentos para representar la secuencia general, los roles y las decisiones ya documentadas; en el paquete se encuentra en diagramas/bpmn/: un BPMN maestro con todos los subprocesos desplegados, los ocho subprocesos (SP-01 a SP-08) por separado y una versión paginada del maestro para imprimir, con fuente editable en draw.io (.drawio), la misma versión en BPMN 2.0 (.bpmn) y exportaciones PDF, SVG y PNG. No lo presentamos como una validación previa del proceso ni como resueltos los asuntos de negocio que no están definidos. Para escribir las historias partimos de la técnica de User Stories. La Tabla 2 muestra cómo aprovechamos cada documento.

Tabla 2. Documentos de la Unidad 2 y su uso en la Unidad 3

Documento	Cómo lo aprovechamos en la Unidad 3
BRD y necesidades BN	Definimos el alcance y justificamos decisiones con reglas existentes, por ejemplo BN-002 a BN-006 y BN-009.
User Stories y criterios CA	Seleccionamos cinco casos de uso y convertimos las condiciones de aceptación en responsabilidades y pruebas.
Glosario	Mantuvimos los mismos nombres para Evento, Contrato, Propuesta, cambio, servicio adicional, saldo, estado y decisión.
Situaciones especiales	Revisamos rutas de rechazo, regularización, devoluciones y sustituciones, sin completar con suposiciones las reglas no definidas.
Matriz de trazabilidad	Relacionamos BN, US/CA, CU, componentes y ADR; dejamos identificadas las relaciones inferidas o pendientes de comprobación.
BPMN de Eventos Nova (elaborado para esta versión)	Nos sirvieron para ver de forma gráfica la secuencia general del proceso, los roles que participan y las decisiones ya documentadas, y como referencia para contrastar los casos de uso y el modelo con el flujo del negocio. La versión final, con el BPMN maestro desplegado, los subprocesos, la fuente editable en draw.io, la versión BPMN 2.0 y las exportaciones PDF, SVG y PNG, está en diagramas/bpmn/.

1.3 Casos de uso elegidos

De todas las historias elegimos cinco casos de uso, CU-01 a CU-05, que cubren el ciclo principal del evento e incluyen un flujo normal y rutas alternativas o de rechazo. No sustituyen todas las historias de la Unidad 2: son un recorte deliberado para poder diseñar y probar con detalle lo más importante.

Tabla 3. Casos de uso elegidos

ID	Caso de uso	Participantes principales	Qué valida
CU-01	Gestionar la propuesta	Cliente y Coordinador	Datos del evento, nuevas versiones, aceptación de la versión vigente o cierre como no aceptada.
CU-02	Formalizar la contratación	Cliente, Coordinador y Administración	Firma, política de no reembolso, anticipo correcto y verificado, y confirmación solo cuando se cumplen los requisitos.
CU-03	Gestionar el cierre del evento	Cliente, Coordinador y Administración	Cancelación procedente o rechazada, regularización de saldo y diferencia entre Cerrado sin contratación, Cancelado y Finalizado.
CU-04	Gestionar cambios en el evento	Cliente, Coordinador, Administración y Proveedor	Viabilidad, anticipación, costos, decisiones requeridas, anexo, pago y aplicación al Evento solo después de confirmar.
CU-05	Gestionar servicios adicionales y proveedores	Cliente, Coordinador, Administración y Proveedor	Plazo de cinco días, disponibilidad, autorizaciones, pago y sustitución sin alterar antes de tiempo la configuración vigente.

2. Comparación de patrones de lógica de negocio

Comparamos Transaction Script, Table Module y Domain Model usando criterios propios del caso. No elegimos por cuál patrón es más moderno ni por cuántas clases genera, sino por las reglas que atraviesan contratos, pagos, cambios, servicios adicionales, proveedores y estados del Evento. La Tabla 4 resume la comparación.

Tabla 4. Comparación de patrones de lógica de negocio

Patrón	Ventajas para el caso	Riesgos o costo en Eventos Nova	Evaluación del equipo
Transaction Script	Resuelve procesos sencillos con procedimientos directos y poco diseño inicial.	Los procedimientos podrían acumular validaciones de plazos, pagos y aprobaciones; la misma regla podría repetirse en varios scripts.	No lo elegimos como patrón principal porque las condiciones cruzan conceptos y estados.
Table Module	Organiza operaciones alrededor de una tabla o conjunto de registros; es práctico cuando el comportamiento sigue la estructura de datos.	Las reglas que involucran varios conceptos exigirían coordinación entre módulos y podrían quedar repartidas.	Lo consideramos, pero no es la opción principal para los procesos más complejos.
Domain Model	Mantiene datos, comportamiento e invariantes cerca de los conceptos del negocio. Facilita reglas sobre versiones, autorización, pagos y transiciones de estado.	Exige definir bien los objetos y sus límites; supone mayor esfuerzo inicial y necesita buenas pruebas.	Elegido y documentado en ADR-001 como patrón principal.

Elegimos Domain Model porque Eventos Nova tiene muchas reglas cruzadas, estados que dependen unos de otros (ciclo de vida del evento, versiones de propuesta, contratos, solicitudes de cambio y obligaciones financieras) y restricciones de tiempo estrictas (días hábiles frente a naturales y ventanas de 24 horas). Esta decisión se apoyó en evidencia de la Unidad 2:

¿Por qué descartamos Transaction Script? Un enfoque procedural acumularía condicionales anidados repartidos en varios scripts, con validaciones duplicadas y errores graves, como confirmar un cambio sin revisar si el evento ya inició o liquidar saldos sin verificación administrativa. ¿**Por qué descartamos Table Module?** Organizar el comportamiento solo por tablas rompe la cohesión del negocio; conceptos como la sustitución asíncrona de proveedores o los conceptos de pago independientes no tienen un mapeo natural en tablas planas. ¿**Por qué adoptamos Domain Model'** Concentra estado, comportamiento e invariantes dentro de entidades y objetos de valor, de modo que ninguna entidad quede en un estado inválido

Como respaldo, ligamos la decisión con BN-002/BN-003 y US-004, US-005 y US-024 (las propuestas requieren versionado y los cambios posteriores dependen de anticipación, viabilidad, costos y aprobación); con BN-004/BN-005 y US-025 (los servicios adicionales, pagos y saldos tienen plazos distintos y no deben confirmarse antes de reunir las condiciones); y con CA-038, CA-041, CA-067, CA-082, CA-094 y CA-095 (los criterios distinguen solicitudes pendientes de confirmadas y la finalización de la cancelación). El costo que aceptamos es una mayor inversión inicial para delimitar el dominio; para no complicarlo de más, no convertimos cada registro en una clase, solo los conceptos con reglas o ciclo de vida relevantes.

3. Mapa de dominio

A partir del glosario y de las reglas definimos un lenguaje ubicuo: los nombres que usa el negocio significan lo mismo en la documentación, el mapa, las clases, los servicios y las pruebas. Con ese lenguaje elaboramos el mapa de dominio editable en Draw.io, con su exportación en PDF, que se muestra en la Figura 1. El mapa distingue raíces de agregado, entidades internas, Value Objects, entidades de referencia, servicios de dominio y Domain Events. La Tabla 5 resume las siete raíces de agregado y la Tabla 6 los demás elementos del modelo

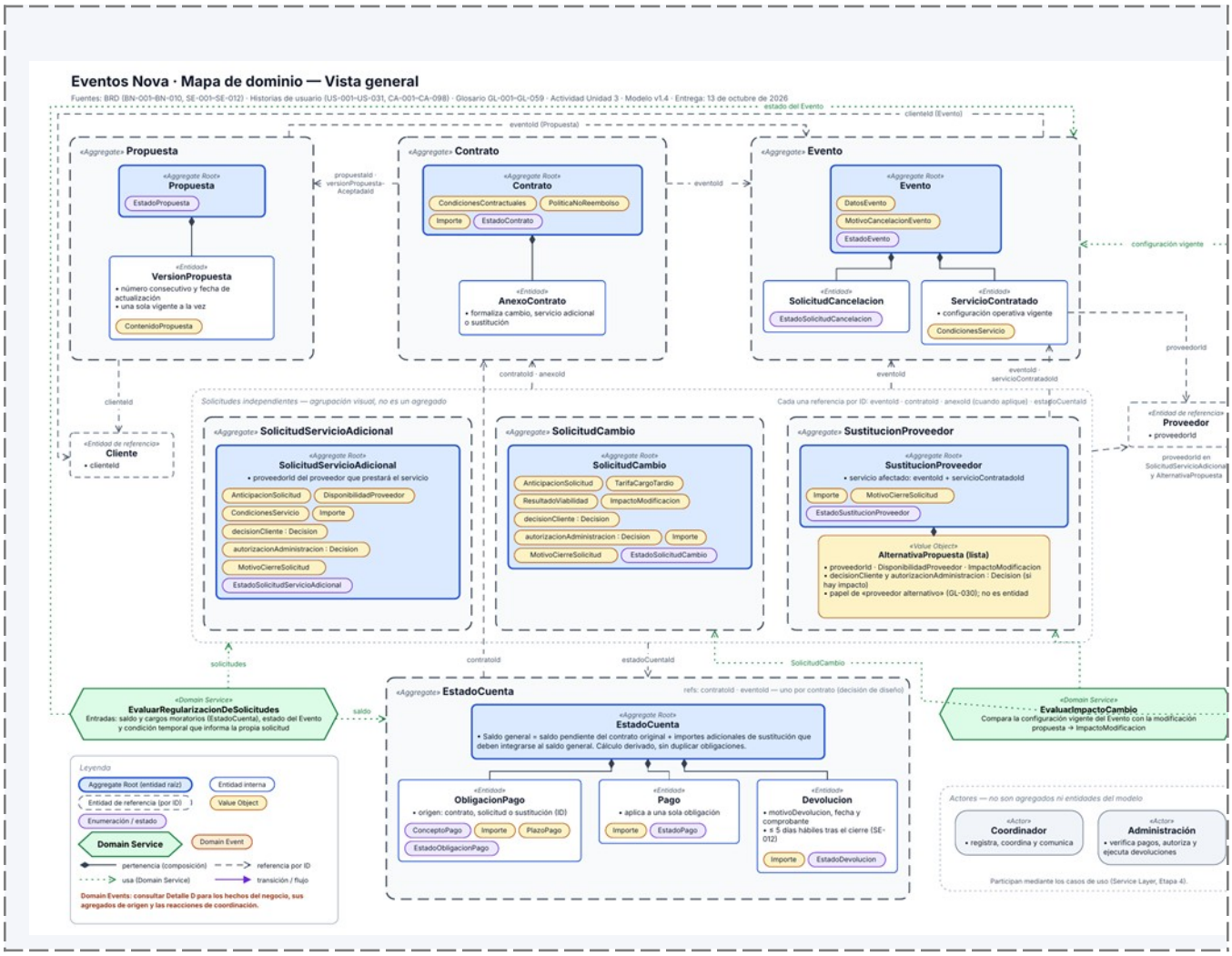


Figura 1. Mapa de dominio de Eventos Nova: vista general.

Tabla 5. Raíces de agregado del modelo

Raíz de agregado	Responsabilidad principal	Ejemplos de datos o reglas protegidas
Propuesta	Gestionar la propuesta y su historial.	VersionPropuesta, versión vigente, aceptación y cierre como no aceptada.
Contrato	Representar las condiciones acordadas y los anexos.	Firma, política de no reembolso, confirmación o cierre sin contratación.
Evento	Conservar el estado y la configuración operativa vigente.	DatosEvento, servicios contratados, inicio, cancelación y finalización.
SolicitudCambio	Controlar un cambio del servicio contratado.	Anticipación, viabilidad, impacto, decisiones, cargo tardío, anexo y pago.
SolicitudServicioAdicional	Gestionar un servicio no incluido originalmente.	Elegibilidad de cinco días, disponibilidad, decisiones, anexo y pago.
SustitucionProveedor	Administrar alternativas cuando un proveedor no puede prestar el servicio.	Alternativas, disponibilidad, impacto y decisiones; el servicio vigente se conserva hasta confirmar.
EstadoCuenta	Controlar obligaciones, pagos y devoluciones del contrato.	ObligacionPago, Pago, Devolucion, saldo original y regularización.

Tabla 6. Otros elementos del modelo

Tipo de elemento	Elementos
Entidades internas	VersionPropuesta, AnexoContrato, ServicioContratado, SolicitudCancelacion, ObligacionPago, Pago y Devolucion.
Value Objects	DatosEvento, Importe, CondicionesServicio, DisponibilidadProveedor, Decision, AnticipacionSolicitud, TarifaCargoTardio, PlazoPago, PoliticaNoReembolso, MotivoCancelacionEvento y MotivoCierreSolicitud, entre otros.
Entidades de referencia	Cliente y Proveedor, representados por identificador porque su ciclo de vida no está dentro del alcance modelado.
Domain Services	EvaluarImpactoCambio y EvaluarRegularizacionDeSolicitudes.
Domain Events	Hechos ya ocurridos, por ejemplo PropuestaAceptada, PagoVerificado, ContratacionConfirmada, CambioConfirmado, ServicioAdicionalConfirmado, EventoIniciado, PlazoRegularizacionVencido y EventoFinalizado.

Las reacciones a los Domain Events no se ejecutan dentro del agregado: las coordina la capa de aplicación después de confirmar la transacción local. Después de la primera versión del mapa lo corregimos: separamos los motivos de cancelación del Evento de los motivos de cierre de una solicitud, y conservamos la elegibilidad temporal que ya había obtenido un servicio adicional cuando la solicitud se pausa por saldo, en lugar de recalcularla sin respaldo.

4. Service Layer y Application Services

Organizamos el código en tres capas: domain, application e infrastructure. La Service Layer es la capa de aplicación: cada servicio coordina un caso de uso, pero no debe apropiarse de las reglas del dominio. Así podemos revisar las reglas del negocio sin amarrarlas a una base de datos, y evitamos concentrar todo en un único servicio gigante. La Tabla 7 resume la responsabilidad de cada capa.

Tabla 7. Responsabilidad de cada capa

Capa	Responsabilidad	Límite o cuidado
Dominio (src/domain)	Modela conceptos, estados, invariantes, rechazos de negocio y hechos ocurridos.	No consulta SQL ni depende de la infraestructura.
Aplicación / Service Layer (src/application)	Coordina CU-01 a CU-05: obtiene agregados, llama a sus métodos, usa interfaces de repositorio, confirma transacciones y devuelve resultados.	No debe concentrar las reglas de negocio; solo coordina.
Infraestructura (src/infrastructure)	Implementa puertos y repositorios en memoria, organiza la composición de dependencias y simula colaboradores externos.	Depende de la aplicación, nunca al revés; no representa persistencia durable.

La Tabla 8 muestra los siete Application Services y las intenciones de negocio que coordinan, tomadas de los métodos que aparecen en el diagrama de la capa de aplicación (Figura 3).

Tabla 8. Application Services y reacciones

Componente	Qué coordina
GestionPropuestaService	Registrar el evento, crear la propuesta, generar nuevas versiones, aceptarla o cerrarla como no aceptada.
FormalizarContratacionService	Preparar el contrato, registrar la comunicación de la política de no reembolso y la firma, dar seguimiento al anticipo pendiente y confirmar la contratación.
GestionPagosService	Obligaciones de anticipo y saldo, solicitud de pago, registro y verificación de pagos, vencimiento del saldo, protocolo de cobro, importe adicional por sustitución, cierre sin realizar y devoluciones.
GestionCierreEventoService	Cancelación solicitada, revisión administrativa, regularización del saldo vencido, registro de la cancelación por falta de regularización, inicio y finalización del evento.
GestionCambiosEventoService	Solicitud de cambio, viabilidad, decisiones del Cliente y de Administración, anexo, confirmación y cierre por falta de pago o por inicio del evento.
GestionServiciosAdicionalesService	Solicitud de servicio adicional, disponibilidad, respuesta de espera, decisiones, anexo, confirmación y cierre por falta de pago o por inicio del evento.
GestionSustitucionProveedorService	Proveedor no disponible, búsqueda de alternativa, decisiones del Cliente y de Administración, anexo, confirmación y cierre por inicio del evento.
Reacciones (ReaccionesEvento, ReaccionesSolicitudes, ReaccionesPagos, ReaccionesNotificacion)	Responden a los Domain Events después del commit, cada una en su propia transacción (por ejemplo, al verificarse un pago o al vencer el plazo de regularización).

5. Repository y Unit of Work

Definimos una interfaz Repository por cada una de las siete raíces. Sus métodos recuperan y guardan agregados completos con identificadores del dominio, sin exponer SQL ni tipos de ORM. Los repositorios que incluye el código son en memoria: las interfaces representan la separación arquitectónica, no una conexión real a una base de datos. La Unit of Work define dónde inicia una transacción, qué cambios se confirman juntos y qué pasa si hay un error. La Tabla 9 resume los componentes principales.

Tabla 9. Componentes de Repository y Unit of Work

Componente	Función	Implementación en el código base
Repository por raíz (7): IPropuestaRepository, IContratoRepository, IEventoRepository, ISolicitudCambioRepository, ISolicitudServicioAdicionalRepository, ISustitucionProveedorRepository, IEstadoCuentaRepository	Obtener y guardar agregados completos (obtenerPorId, guardar y consultas como listarPorEventoId o listarPendientesPorEventoId).	Una clase en memoria por interfaz, por ejemplo PropuestaRepositoryMemoria y EstadoCuentaRepositoryMemoria.
IUnitOfWork	Iniciar, registrar, confirmar o revertir los cambios de una transacción.	UnitOfWorkMemoria.
IPublicadorEventosDominio y Outbox	Guardar los Domain Events junto con el commit y ejecutar después las reacciones, cada una en su transacción.	PublicadorEnProceso y DespachadorOutbox, en memoria (propuesta de ADR-004).
Puertos externos	Consultar disponibilidad de proveedores, notificar, verificar pagos y ejecutar devoluciones.	Versiones simuladas o en memoria (ConsultaDisponibilidadSimulada, NotificadorEnMemoria, VerificadorPagosSimulado y EjecutorDevolucionesSimulado).

Las reglas de transacción que definimos son estas: las operaciones de origen modifican una raíz por transacción, con la excepción documentada de comunicarSolicitudPago(); los Domain Events se guardan con el commit; y las reacciones corren en transacciones propias. Para reducir el riesgo de perder eventos después del commit propusimos un Outbox, pero lo presentamos solo como propuesta: en el código vive en memoria y no sobrevive a un reinicio. Durante la revisión corregimos un caso en que una reacción confirmaba dos agregados en una sola transacción y una fila duplicada en nuestra tabla de Unit of Work.

6. CQRS

Aplicamos un CQRS básico: separamos Commands y Queries, pero sin dividir la base de datos. Los comandos cambian el estado y delegan en la Service Layer; las consultas solo leen y construyen View Models, y los Query Handlers no usan la Unit of Work. La Tabla 10 muestra los componentes documentados en el diagrama de clases de la Figura 4.

Tabla 10. Componentes principales de CQRS

Tipo	Componentes	Responsabilidad
Command Handlers	GenerarNuevaVersionPropuestaHandler, ConfirmarCambioHandler, DarSeguimientoAnticipoPendienteHandler y RegistrarCancelacionPorFaltaDeRegularizacionHandler.	Reciben el Command y delegan en el Application Service correspondiente; modifican el estado.
Query Handlers	ConsultarEstadoPagosHandler y ConsultarSolicitudesPendientesHandler.	Solo leen; la consulta de pagos recibe el eventoId y consulta las cuentas correspondientes.
View Models	EstadoPagosViewModel, EstadoCuentaResumenViewModel, SolicitudesPendientesViewModel y SolicitudPendienteViewModel.	Modelos de lectura; no deben convertirse en objetos de escritura ni duplicar datos financieros que pertenecen a EstadoCuenta.

En esta etapa la IA propuso un SolicitudPendienteViewModel que incluía datos financieros que correspondían al estado de pagos; lo ajustamos para no duplicar esa información.

7. Revisión de los principios SOLID

Revisamos las etapas 3 a 6 con los cinco principios SOLID y registramos cada hallazgo con su evidencia, impacto y corrección o justificación. La Tabla 11 resume lo que encontramos. No todos los hallazgos fueron errores: H-05 y H-07 fueron decisiones positivas que conservamos y documentamos.

Tabla 11. Hallazgos de la revisión SOLID y acciones

Principio	Hallazgo del proyecto	Acción o estado
S — Responsabilidad única	H-01, H-02, H-03, H-08, H-09 y H-10 detectaron métodos con responsabilidades mezcladas, reacciones o avisos duplicados y la cancelación asignada al componente equivocado.	Separamos responsabilidades: el vencimiento informa y el Coordinador registra la cancelación.
O — Abierto/cerrado	H-04 afectaba el manejo de reglas y condiciones temporales en solicitudes.	Corrección aplicada en el Domain Service y en las solicitudes.
L — Sustitución de Liskov	H-05 registró una decisión positiva sobre la sustitución de implementaciones.	La conservamos y documentamos; debe entenderse en el marco de las interfaces propuestas.
I — Segregación de interfaces	H-06 encontró información o dependencias innecesarias entre View Models, Handler e interfaces.	Ajustamos View Models e interfaces de aplicación.
D — Inversión de dependencias	H-07 confirmó la dirección de dependencias como decisión positiva y se corrigieron dos dependencias concretas.	Dominio y aplicación se separan de los detalles de infraestructura; la implementación sigue siendo un prototipo en memoria.

Con esta revisión entendimos que SOLID sirve para encontrar errores reales, no solo para cumplir una lista. Además, las correcciones técnicas las comprobamos contra el código y con pruebas nuevas, y las reglas que la documentación no define las dejamos pendientes en lugar de inventarlas.

8. Diagramas de clases, secuencia y decisiones ADR

Representamos el modelo en varias vistas para poder comparar el mapa de dominio con las clases, las dependencias y el orden en el tiempo de los procesos. Los diagramas de clases y de secuencia tienen fuente PlantUML y exportación; el mapa tiene fuente Draw.io. Los espacios siguientes están reservados para pegar las imágenes.

8.1 Diagramas de clases

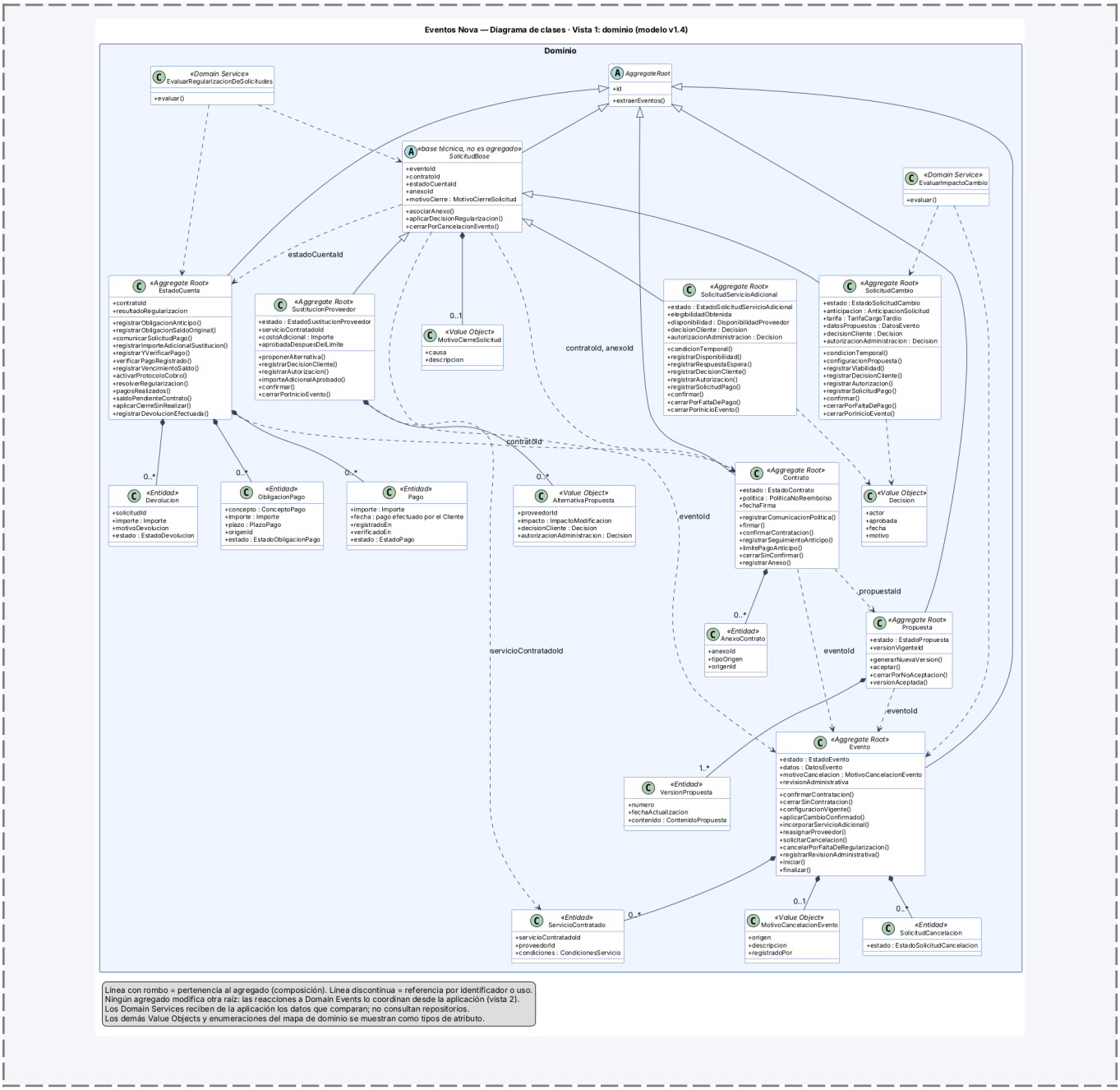


Figura 2. Diagrama de clases, vista del dominio (clases_1_dominio.puml).

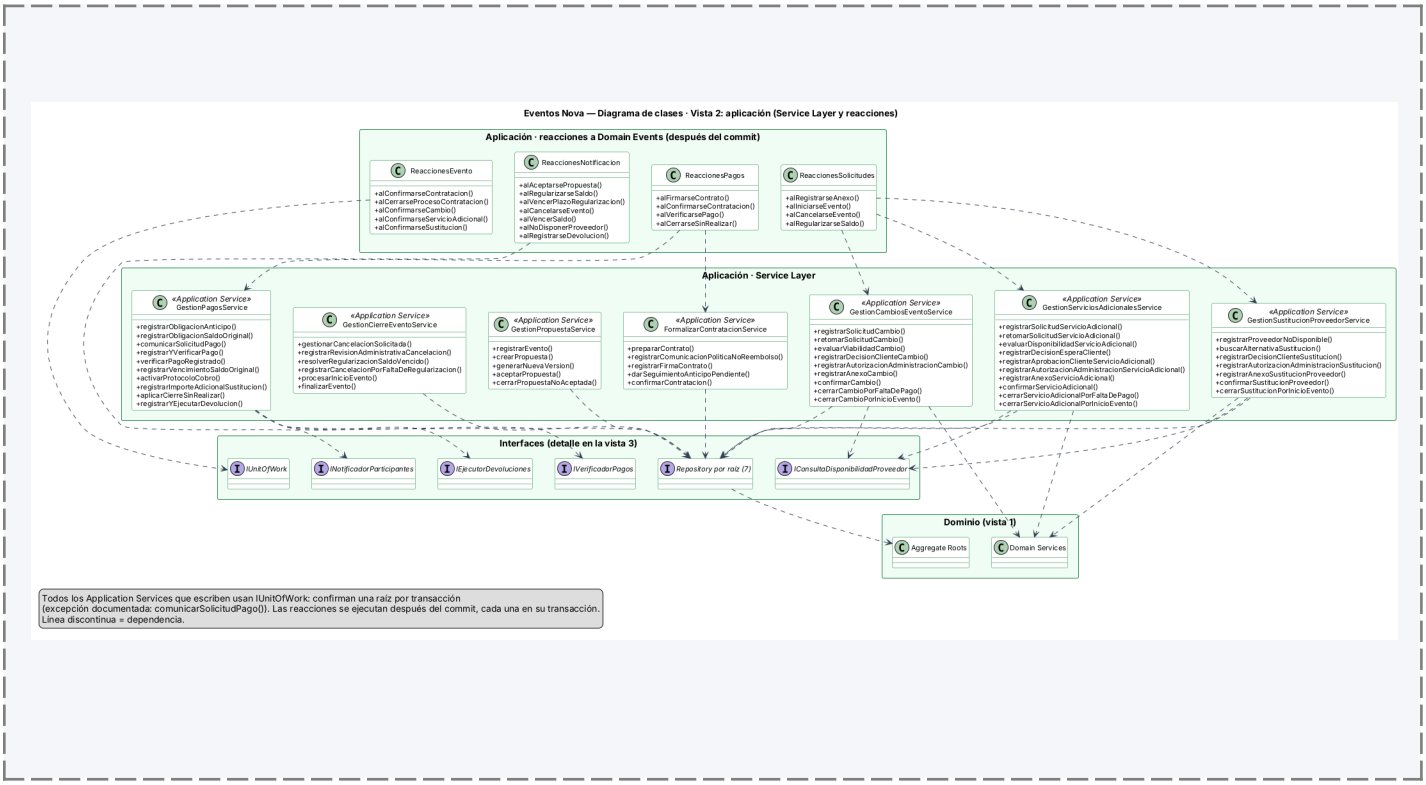


Figura 3. Diagrama de clases, vista de aplicación (clases_2_aplicacion.puml).

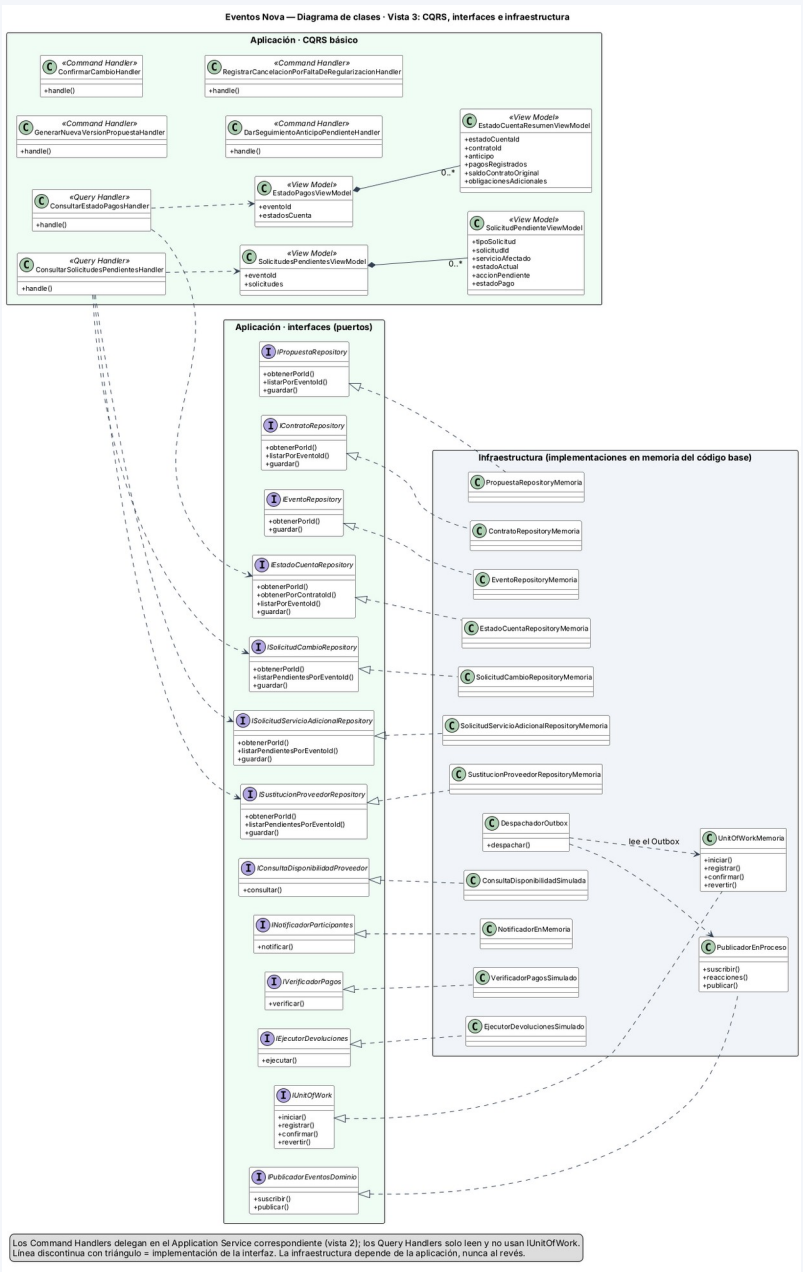


Figura 4. Diagrama de clases, CQRS e infraestructura (clases_3_cqrs_infraestructura.puml).

8.2 Diagramas de secuencia

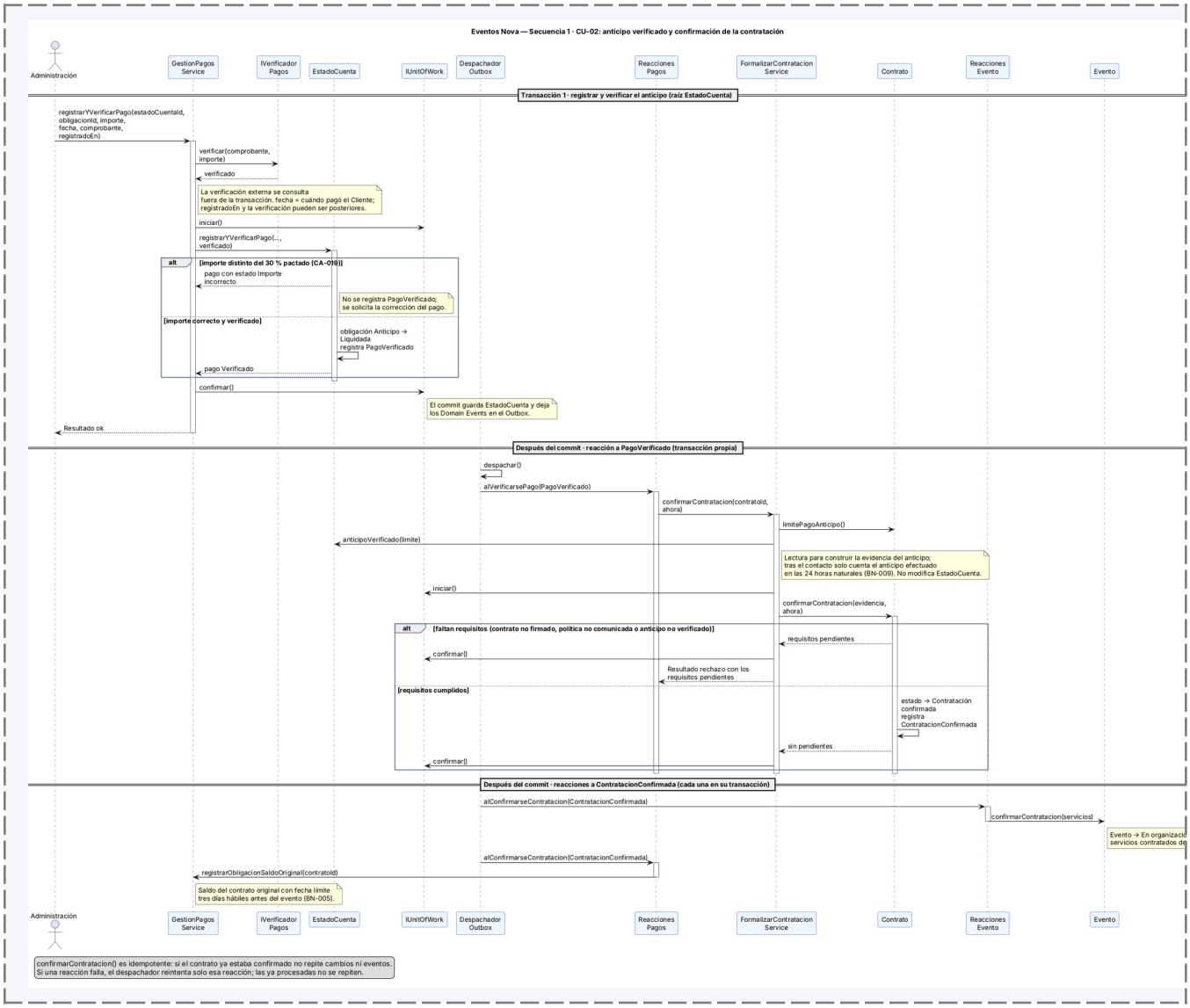


Figura 5. Diagrama de secuencia de CU-02, confirmación de contratación (secuencia_1_confirmacion_contratacion.puml).

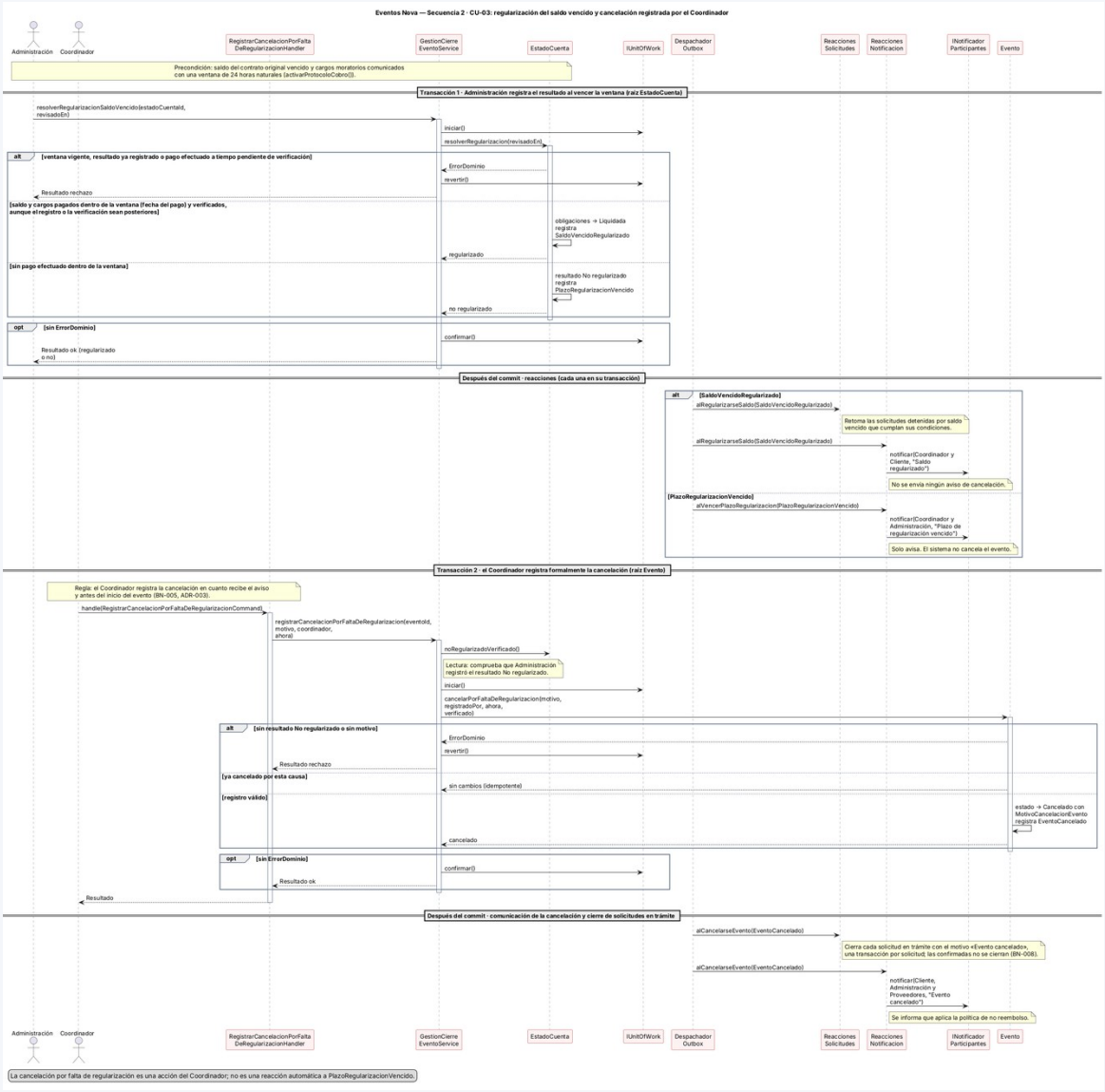


Figura 6. Diagrama de secuencia de CU-03, regularización y cancelación (secuencia_2_regularizacion_y_cancelacion.puml).

8.3 Decisiones arquitectónicas (ADR)

Documentamos cuatro Architecture Decision Records con título, estado, contexto, alternativas, decisión, consecuencias y trazabilidad. Es importante distinguir una decisión aceptada de una propuesta pendiente, por eso respetamos el estado real de cada uno en la Tabla 12.

Tabla 12. Resumen de los ADR y su estado

ADR	Decisión	Estado	Consecuencias y pendientes
ADR-001	Domain Model como patrón principal de lógica de negocio, con agregados que protegen sus invariantes.	Aceptada	Reglas localizadas y verificables con pruebas por agregado. Costo: mayor esfuerzo inicial. Riesgo: límites de agregado mal definidos (se mitigó en la Etapa 3). Validación con el BPMN: modelo contrastado con el BPMN maestro desplegado y SP-01 a SP-08; el aviso del proveedor recibido después del inicio queda fuera del alcance (SP-05).
ADR-002	Mantener SustitucionProveedor y EstadoCuenta como raíces de agregado propias (siete raíces en total), con referencias por identificador.	Aceptada	El Evento no cambia hasta SustitucionProveedorConfirmada y las reglas financieras se protegen juntas. Costo: más repositorios y reacciones. Un EstadoCuenta por contrato; las consultas por evento usan listarPorEventoId().
ADR-003	La cancelación por falta de regularización la registra el Coordinador; PlazoRegularizacionVencido solo avisa.	Aceptada	Respeta las responsabilidades del negocio y está probada. El Coordinador la registra en cuanto recibe el aviso y antes del inicio del Evento (P-08 resuelto). Se distingue del cierre por anticipo no recibido (Evento Cerrado sin contratación).
ADR-004	Reacciones a Domain Events después del commit, en transacciones propias, con Outbox propuesto.	Aceptada (Outbox en memoria en el prototipo)	Cada reacción se reintenta sin duplicar efectos (comprobado con pruebas sobre el Outbox en memoria). Costo: consistencia eventual e idempotencia. Lo confirmado antes del inicio se aplica aunque la reacción llegue después (O-02 resuelta); con el Evento cancelado no se aplica y queda en «Revisión» (BN-008). Trabajo posterior: Outbox persistente en la misma base de datos que los agregados.

Figura 9. Capturas de ADR.

ADR-001: Domain Model como patrón principal de lógica de negocio

Campo del ADR	ADR-001
Título	Domain Model como patrón principal de lógica de negocio
Estado	Aceptada (decisión de la Etapa 2, conservada por el equipo como base aprobada).
Contexto	Las reglas relacionan propuestas, contratos, eventos, solicitudes, proveedores y pagos, con estados, plazos y aprobaciones que deben cumplirse antes de confirmar una operación (BN-002 a BN-005, BN-

	009).
Alternativas	Transaction Script: sencillo, pero acumularía validaciones de pagos, plazos y autorizaciones en procedimientos extensos. Table Module: organiza operaciones por tabla, pero las reglas que cruzan varios conceptos requerirían coordinación entre módulos.
Decisión	Domain Model con agregados que protegen sus invariantes, porque permite ubicar cada regla en el concepto responsable y evitar confirmaciones inválidas.
Consecuencias	Beneficio: reglas localizadas y verificables con pruebas por agregado. Costo: mayor esfuerzo inicial de diseño. Riesgo: límites de agregado mal definidos; se mitigó en la Etapa 3. Validación con el BPMN: el modelo se contrastó con el BPMN maestro completamente desplegado y con los subprocesos SP-01 a SP-08 (diagramas/bpmn). Cada raíz corresponde a un tramo del proceso: Propuesta (SP-01); Contrato y EstadoCuenta (SP-02, SP-06 y SE-012, SP-08); Evento (inicio, ejecución, finalización y cancelación, SP-07); SolicitudCambio (SP-03); SolicitudServicioAdicional (SP-04) y SustitucionProveedor (SP-05). El BPMN comprueba el estado del Evento al recibir el aviso del proveedor: el aviso recibido después del inicio quedó definido como fuera del alcance del sistema y, con el Evento cancelado, no se confirma ninguna sustitución (BN-008), lo que el prototipo respeta.
Trazabilidad	BN-002, BN-003, BN-004, BN-005, BN-006, BN-009; US-004, US-005, US-008, US-024, US-025; CA-007 a CA-010, CA-015 a CA-019, CA-038, CA-041, CA-067, CA-082; CU-01 a CU-05.

ADR-002: SustitucionProveedor y EstadoCuenta como raíces de agregado independientes

Campo del ADR	ADR-002
Título	SustitucionProveedor y EstadoCuenta como raíces de agregado independientes
Estado	Aceptada (decisión pendiente cerrada en la revisión de consistencia; se conserva la arquitectura existente).
Contexto	Una sustitución de proveedor es un proceso largo con alternativas sucesivas, y el servicio original debe seguir vigente hasta confirmar el reemplazo. Las reglas de saldo general, regularización y devoluciones abarcan varias obligaciones del mismo contrato.
Alternativas	Sustitución como entidad de Contrato o de Evento: mezclaría el documento o la configuración con un proceso pendiente y bloquearía el Evento. Reutilizar SolicitudCambio: la inicia el proveedor y sus aprobaciones dependen del impacto. Una raíz por obligación de pago: las reglas que cruzan obligaciones solo podrían verificarse fuera del límite.
Decisión	Mantener SustitucionProveedor y EstadoCuenta como raíces propias (siete raíces en total), con referencias por identificador.
Consecuencias	Beneficio: Evento no cambia hasta SustitucionProveedorConfirmada; las reglas financieras se protegen juntas. Costo: más repositorios y reacciones a Domain Events. Decisión arquitectónica asociada: un

	EstadoCuenta por contrato, por lo que las consultas por evento usan listarPorEventoId().
Trazabilidad	SE-001, SE-012, BN-004, BN-005, BN-006; US-011, US-021, US-031; GL-056, GL-058, GL-059; CA-026, CA-046 a CA-054, CA-096 a CA-098.

ADR-003: La cancelación por falta de regularización la registra el Coordinador

Campo del ADR	ADR-003
Título	La cancelación por falta de regularización la registra el Coordinador
Estado	Aceptada (regla indicada por el equipo y documentada en BN-005, BN-008 y CA-027).
Contexto	En una revisión anterior la reacción a PlazoRegularizacionVencido cancelaba el Evento automáticamente. Las fuentes indican que Administración revisa los pagos e informa al Coordinador, y que este registra la cancelación, indica el motivo e informa a los involucrados.
Alternativas	Cancelación automática por reacción: contradice la regla y omite el motivo del Coordinador. Tarea manual sin Domain Event: perdería el aviso trazable del vencimiento.
Decisión	Conservar PlazoRegularizacionVencido como hecho verificado por Administración; su reacción solo avisa. El Coordinador ejecuta registrarCancelacionPorFaltaDeRegularizacion() mediante su Command; Evento exige la verificación previa y el motivo.
Consecuencias	Beneficio: responsabilidades del negocio respetadas y probadas. El Coordinador registra la cancelación en cuanto recibe el aviso de Administración y antes del inicio del Evento; después del inicio ya no procede y el adeudo se conserva (P-08 resuelto). Este escenario se distingue del cierre por anticipo no recibido, que deja el Evento Cerrado sin contratación.
Trazabilidad	BN-005, BN-008, SE-007, SE-010 (distinción del cierre sin contratación); CU-02, CU-03; US-010, US-011; CA-020, CA-021, CA-025, CA-027; hallazgo H-10; P-08 resuelto.

ADR-004: Reacciones a Domain Events después del commit, en transacciones propias, con Outbox propuesto

Campo del ADR	ADR-004
Título	Reacciones a Domain Events después del commit, en transacciones propias, con Outbox propuesto
Estado	Aceptada (en el prototipo el Outbox se implementa en memoria; el Outbox persistente queda como trabajo posterior).
Contexto	Confirmar una operación afecta otros agregados (por ejemplo, ContratacionConfirmada cambia Evento y EstadoCuenta). Modificar varias raíces en el método de origen rompería los límites de agregado; publicar antes del commit comunicaría hechos que podrían revertirse.
Alternativas	Una sola transacción con todos los agregados: acoplamiento y bloqueos. Publicación en memoria

	después del commit sin registro: se pierden eventos si el proceso falla. Bus de mensajes: desproporcionado para el alcance.
Decisión	Cada operación de origen confirma su raíz (excepción justificada: comunicarSolicitudPago()). Los Domain Events se guardan con el commit en un Outbox y un despachador ejecuta reacciones idempotentes, cada una en su transacción.
Consecuencias	Beneficio: cada reacción se reintenta sin repetir las ya procesadas y sin duplicar efectos (comprobado con pruebas sobre el Outbox en memoria). Costo: consistencia eventual entre agregados y necesidad de idempotencia. Riesgo: en el código base el Outbox vive en memoria, por lo que no sobrevive a un reinicio. Trabajo posterior: Outbox persistente en la misma base de datos que los agregados. Una operación confirmada antes del inicio del Evento se aplica aunque su reacción llegue después del inicio (O-02 resuelta). Si el Evento se canceló antes de aplicarla, la operación no se aplica, la configuración no cambia y la solicitud conserva su confirmación; el registro del Outbox queda en «Revisión» con el motivo (BN-008).
Trazabilidad	BN-006, BN-009, SE-012; US-008, US-010, US-021; CA-019, CA-021, CA-051, CA-070, CA-071, CA-096; hallazgos H-01, H-02, H-03.

9. Código base y estructura de carpetas

Implementamos una base en Python para que la arquitectura se pueda revisar. La estructura reproduce las tres capas y deja los colaboradores externos detrás de adaptadores o simulaciones. La Tabla 13 resume el contenido de cada carpeta y lo que nos permite comprobar.

Tabla 13. Estructura de carpetas del código base

Carpeta	Contenido observado	Qué permite comprobar
src/domain/	Modelos de Propuesta, Contrato, Evento, EstadoCuenta, solicitudes, valores, servicios y eventos de dominio.	Reglas, estados y decisiones de las raíces; sus métodos pueden probarse sin interfaz.
src/application/	Application Services, puertos, CQRS, reacciones y resultados.	Coordinación de casos de uso, separación de consultas y comandos, y reacciones a hechos del dominio.
src/infrastructure/	Adaptadores, composición de dependencias, repositorios en memoria y Unit of Work.	Concreta las interfaces para el prototipo; no representa persistencia durable.
tests/	Seis módulos de pruebas y la carpeta tests/evidencias/.	Flujos, invariantes, transacciones, reacciones, reglas corregidas y coherencia entre documentos y diagramas.

El paquete del proyecto también incluye la documentación, los diagramas y el README con el alcance y las instrucciones de ejecución. La Tabla 14 resume los entregables de la actividad.

Tabla 14. Entregables de la actividad

Ruta	Entregable disponible
docs/informe/	Informe de arquitectura en DOCX y PDF.
docs/ADR/	Cuatro ADR en DOCX y PDF, con estados diferenciados.
docs/trazabilidad/	Matriz de trazabilidad en XLSX y su exportación PDF.
docs/bitacora ia/	Bitácora de IA editable y PDF.
docs/unidad 2/	BRD corregido, User Stories con criterios Gherkin y glosario corregido.
diagramas/	Mapa en Draw.io/PDF, tres vistas de clases y dos secuencias, cada una con fuente PlantUML y exportaciones, y el BPMN.
diagramas/bpmn/	BPMN maestro con los subprocesos desplegados y subprocesos SP-01 a SP-08: fuente editable en draw.io (.drawio), versión BPMN 2.0 (.bpmn) y exportaciones PDF, SVG y PNG; versión paginada del maestro (A3) para imprimir.
src/ y tests/	Código Python en tres capas, seis módulos de pruebas, log de ejecución y evidencia visual.
README.md	Descripción de la empresa y el alcance, estructura, instrucciones y listado de archivos.
Repositorio en GitHub	github.com/leslynieto64-del/eventos_nova_equipo16_CLNEG_004 (enlace al final del reporte). Falta verificar el historial de commits.

10. Validación, pruebas automatizadas y limitaciones

10.1 Entorno y comando de ejecución

Las pruebas se ejecutaron con unittest, la biblioteca estándar de Python. Se corrieron en una terminal bash de Linux (WezTerm) desde la carpeta raíz del proyecto, que contiene src/ y tests/. La Tabla 15 reúne los datos documentados.

Tabla 15. Entorno de ejecución documentado en la evidencia

Elemento	Valor documentado
Sistema operativo	Linux (Ubuntu 24.04)
Editor	Terminal WezTerm (bash)
Lenguaje	Python 3.13.16
Marco de pruebas	unittest
Carpeta raíz	EVENTOS_NOVA_U3_ARQUITECTURA_CLNEG
Comando	python -m unittest discover -s tests -v (la salida completa se conserva en ejecucion-pruebas.txt)
Ejecución final registrada	Ran 91 tests in 0.950s, OK; misma ejecución en ejecucion-pruebas.txt y en la Figura 10

10.2 Qué cubre la suite y resultados

La carpeta tests contiene seis módulos. Las pruebas verifican los comportamientos programados en los escenarios que cubre la suite: reglas de negocio del dominio, coordinación de los casos de uso.

Límites transaccionales, reacciones a Domain Events y coherencia entre diagramas, documentos y código. La Tabla 16 muestra la cantidad de pruebas por módulo.

Tabla 16. Módulos de prueba y cobertura principal

Módulo de prueba	Pruebas	Cobertura principal
test_coherencia_diagramas.py	14	Que los métodos, clases, estados y Domain Events del mapa existan en el código; que los documentos citen métodos existentes y usen una sola versión del modelo; que la matriz de trazabilidad no tenga caracteres invisibles ni identificadores mal formados; y que los diagramas de clases y de secuencia correspondan al código.
test_correcciones_etapas_3_a_7.py	12	Correcciones de avisos de regularización, aplicación del cambio confirmado al Evento, condiciones del servicio adicional, sustitución tardía de proveedor, revisión administrativa de la cancelación, solicitudes en trámite con saldo vencido y rechazos devueltos como resultado.
test_etapa9_casos_de_uso.py	24	Recorrido de CU-01 a CU-05 con flujos principales, alternativos y de error: propuesta, contratación, cancelación y cierre del evento, cambios, servicios adicionales y sustitución de proveedores.
test_etapa9_transacciones_y_reacciones.py	10	Límites transaccionales (una raíz por transacción, reversión ante fallas, publicación después del commit), reacciones a Domain Events (reintentos e idempotencia) y dependencias entre capas.
test_reglas_corregidas.py	19	Cancelación por saldo vencido registrada por el Coordinador, anticipo no recibido, plazo de cinco días de los servicios adicionales y pago efectuado a tiempo frente a su registro y verificación posteriores.
test_decisiones_stakeholder.py	12	Decisiones del stakeholder de la revisión de consistencia: cancelación por saldo vencido antes del inicio con el adeudo conservado, ninguna confirmación con el Evento cancelado, cierre de las solicitudes en trámite con el motivo «Evento cancelado», operación confirmada no aplicada con el Evento cancelado y reprogramación de la fecha fuera del alcance.
Total	**91**	**91 aprobadas; 0 fallidas, 0 errores y 0 omitidas.**

La ejecución final de esta versión, registrada en ejecucion-pruebas.txt y en la captura de la Figura 10, terminó con Ran 91 tests in 0.950s y OK: 91 pruebas ejecutadas, 91 aprobadas, 0 fallidas, 0 errores y 0 omitidas, como muestra la Tabla 17. Ejecuciones anteriores: la versión con 82 pruebas terminó con OK en 0.982 s y la de 70 pruebas, en 0.601 s.

Tabla 17. Resultados documentados de las pruebas

Indicador	Evidencia con captura
Pruebas ejecutadas	91
Pruebas aprobadas	91
Pruebas fallidas	0
Pruebas con error	0
Pruebas omitidas	0
Tiempo registrado	0.950 s
Resultado final	OK

En unittest, “OK” sin anotaciones adicionales indica que no hubo fallas, errores ni pruebas omitidas.

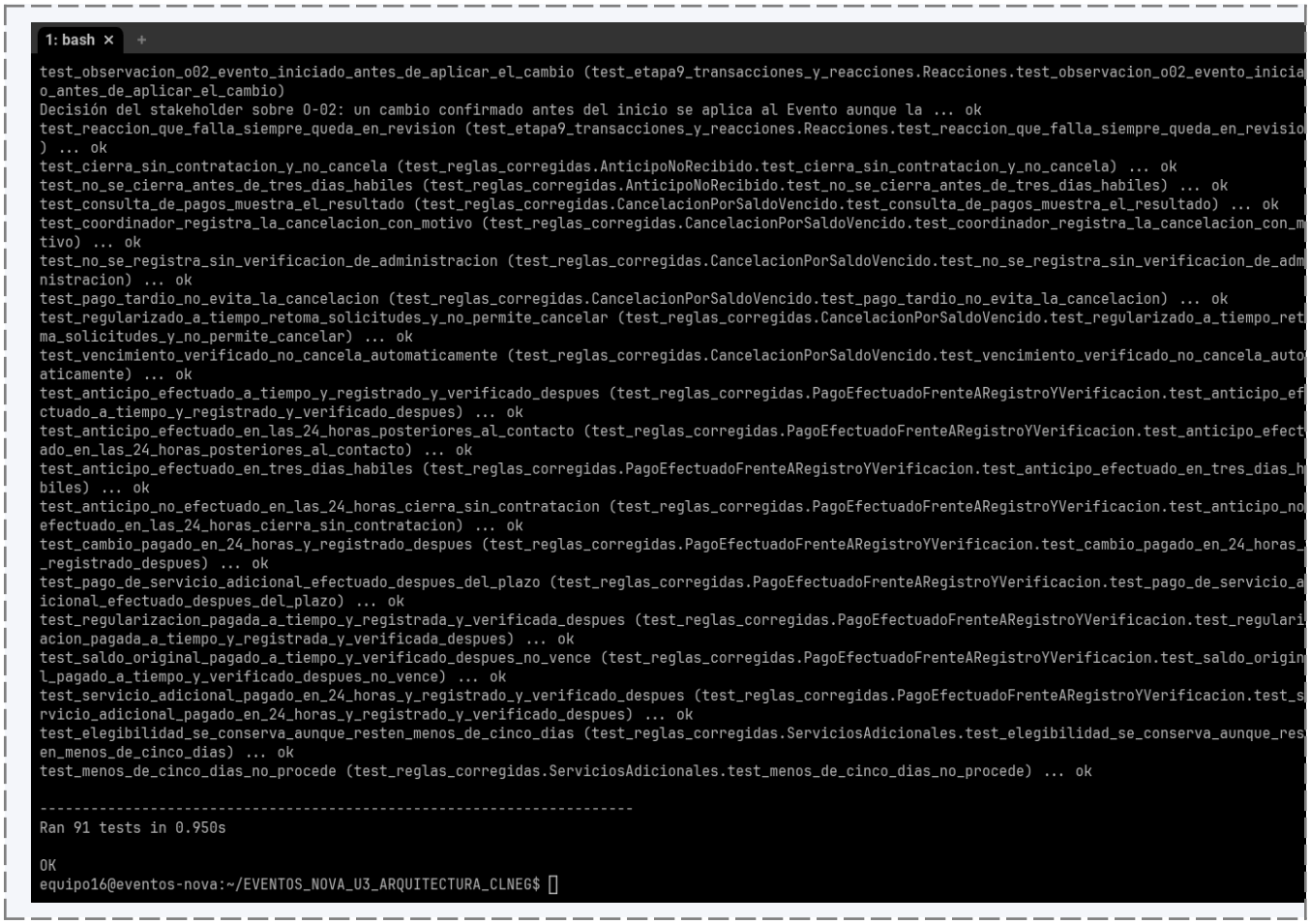


Figura 10. Captura de la ejecución de las pruebas automatizadas de Eventos Nova (ejecución final: 91 pruebas, OK).

10.3 Defectos corregidos

La documentación de pruebas registra los defectos D-01 a D-07, que corregimos: impedir liquidar un saldo antes de la verificación administrativa; aplicar una sola vez los cambios confirmados; conservar condiciones aprobadas del servicio adicional; evitar reutilizar plazos vencidos en sustituciones; registrar la revisión administrativa; impedir avisos y reacciones duplicadas; y devolver los rechazos de negocio como resultados y no como fallas técnicas. Las primeras pruebas también tuvieron errores en nombres e importaciones y un escenario no alcanzable. En una primera ejecución falló la prueba test_documentos_sin_nombres_de_archivos_fuente porque la búsqueda de documentos Word incluía la carpeta tests/evidencias/, cuyo texto menciona extensiones de archivos de Word. Ajustamos la búsqueda para excluir esa carpeta y volvimos a ejecutar toda la suite; la corrección no cambió la lógica de negocio. En la revisión de consistencia posterior la misma prueba falló porque este reporte también citaba esa extensión, y se corrigió la redacción. En esa revisión se corrigieron además dos defectos: una solicitud podía confirmarse con el Evento cancelado (D-08) y un cambio podía reprogramar la fecha del Evento, que está fuera del alcance (D-09). En la corrección final se incorporó la regla de cerrar con el motivo «Evento cancelado» las solicitudes en trámite cuando se cancela el Evento (BN-008).

10.4 Limitaciones y pendientes

Una suite en verde demuestra solo los comportamientos que cubren sus aserciones. No demuestra que exista una base de datos real, una persistencia duradera ni integraciones externas operativas, porque los repositorios, la Unit of Work y el Outbox usados en las pruebas son en memoria, y la consulta de disponibilidad de proveedores, la verificación de pagos, las devoluciones y las notificaciones están simuladas. Tampoco acreditan concurrencia, rendimiento, una interfaz gráfica completa ni un sistema listo para producción. La Tabla 18 reúne lo que sigue pendiente.

Tabla 18. Observaciones y pendientes abiertos

ID	Pendiente	Estado
O-01	Orden de evaluación entre el plazo de cinco días del servicio adicional y el saldo vencido: primero el evento, después el plazo y al final el saldo; se registra solo el primer motivo (SE-004).	Resuelta por decisión del negocio.
O-02	Si el Evento inicia entre la confirmación de un cambio y su reacción, el cambio confirmado se aplica igualmente al Evento (BN-006). Si el Evento se canceló antes, no se aplica y la solicitud conserva su confirmación (BN-008).	Resuelta por decisión del negocio.
P-08	El Coordinador registra la cancelación en cuanto recibe el aviso de Administración y antes del inicio del evento (BN-005).	Resuelto (ADR-003).
P-01 a P-07	P-04 y P-06 se resolvieron (BN-008 y SE-007) y P-05 quedó cubierto al dejar fuera del alcance la reprogramación de la fecha. P-01, P-02, P-03 y P-07 no tienen una definición verificable en los documentos.	Sin regla nueva para los puntos no verificables.
ADR-002 y ADR-004	Aceptadas; el Outbox persistente en una base de datos real queda como trabajo posterior.	Trabajo posterior.
Repositorio GitHub	Verificar el historial de commits del repositorio.	Pendiente de verificar.
Ejecución de las	Versión de Python y comando de esa ejecución.	Completado: Python

pruebas		3.13.16, python -m unittest discover -s tests -v; Ran 91 tests in 0.950s, OK.

11. Uso de inteligencia artificial

11.1 Cómo usamos Claude y cómo revisamos sus respuestas

Usamos Claude (Anthropic) como apoyo en cada etapa. Nos repartimos las etapas: Lesly trabajó los casos de uso, los patrones, el mapa de dominio y las revisiones finales de plazos y trazabilidad; Christopher, la Service Layer, el Repository con la Unit of Work y CQRS; y Francisco, SOLID, el código base, los diagramas con los ADR y las pruebas. En cada una seguimos casi el mismo ciclo: preparamos el contexto con el BRD, las User Stories, los criterios y el glosario; pedimos una propuesta indicando que no agregara nada sin respaldo; comparamos la respuesta con nuestros documentos; corregimos los errores o supuestos; validamos con revisión y pruebas; y registramos todo en la bitácora de IA.

Tratamos a la IA como una propuesta, no como una respuesta final. La IA sugería alternativas y borradores; el equipo decidía qué conservar, qué corregir y qué dejar pendiente. La Tabla 19 resume las más relevantes. Reconocemos que el historial original no conserva todos los detalles de las revisiones y que, en algunos casos, no podemos asegurar que la primera respuesta de la IA hubiera detectado los problemas que encontramos después.

Tabla 19. Bitácora de IA: propuestas, errores y decisiones del equipo

Iteracion	Objetivo	Prompt o enlace	Respuesta util	Error o supuesto	Decision del equipo
1 ·	Etapa 1: Definir los casos de uso y sus flujos principales, alternativos y de rechazo.	(Lesly): «Con el BRD, las US y sus CA, identifica los casos de uso que cubren el ciclo del evento. Para cada uno indica actor, objetivo, entradas, reglas, resultado y flujos alternativos, vinculándolos con BN, US y CA».	Se consolidaron CU-01 a CU-05 con escenarios normales y excepciones. En revisiones posteriores se detectaron puntos que debían aclararse entre documentos, como el momento de inicio del evento y las reglas de cancelación.	Los cinco casos de uso cubrían el proceso completo. En revisiones posteriores se encontraron reglas y excepciones que necesitaban aclararse entre documentos; no podemos asegurar que la primera respuesta las hubiera detectado.	Se conservaron los cinco casos de uso como punto de partida. Después se revisaron contra las reglas y los criterios para corregir contradicciones y completar los flujos que lo necesitaban.
2 ·	Etapa 2: Comparar patrones de lógica de negocio y justificar la selección.	(Lesly): «Compara Transaction Script, Table Module y Domain Model para Eventos Nova. Considera estados, plazos, aprobaciones y reglas que cruzan contratos, pagos, solicitudes y proveedores. Presenta ventajas, riesgos y evidencia de los casos de uso, y recomienda el patrón más adecuado».	La comparación condujo a seleccionar Domain Model como patrón principal por la complejidad de las reglas y las transiciones de estado.	Elegir el patrón solo por sus ventajas generales, sin relacionarlo suficientemente con los casos de uso y sus reglas.	Se eligió Domain Model como patrón principal y la decisión quedó registrada en ADR-001. Las capas de aplicación, persistencia y consultas se definieron por separado en las etapas siguientes.
3	Etapa 3: Representar visualmente los conceptos principales del negocio y sus relaciones.	(Lesly): «Convierte la arquitectura de dominio validada de Eventos Nova en un mapa de dominio editable en Draw.io. Usa como base el lenguaje ubicuo, las entidades, los Value Objects, los siete Aggregates con sus raíces, los Domain Services y los Domain Events definidos en la documentación. No agregues clases, reglas ni relaciones que no estén respaldadas por el	Se generó un mapa de dominio con los elementos principales del modelo, sus relaciones y vistas de detalle para facilitar la lectura.	Fue necesario revisar que las relaciones y clasificaciones del diagrama coincidieran con la documentación y que la vista general no quedara demasiado saturada.	Se conservó el mapa como representación visual del modelo y se revisaron sus detalles para mejorar la claridad. Los elementos o relaciones que no podían justificarse con la documentación debían corregirse o quedar pendientes.

		modelo. Organiza el diagrama para que sea legible, distingue los elementos por tipo y conserva los nombres utilizados en los documentos. Entrega el archivo editable y una exportación en PDF».			
3.1	Etapa 3: Corregir el mapa de dominio, el lenguaje ubicuo, los agregados y los servicios de dominio.	(Lesly): «Revisa el mapa contra BN-004, BN-005, SE-004, CA-072 y CA-080. La elegibilidad de un servicio adicional no debe perderse por una pausa de saldo. Evalúa separar los motivos de cancelación/cierre, conserva las decisiones que tengan sustento y marca lo no definido como pendiente. No afirmes haber revisado el BPMN porque no está disponible».	Se revisaron las siete raíces y los dos Domain Services; se identificó el riesgo de mezclar la cancelación del Evento con el cierre de solicitudes y se documentaron pendientes P-01 a P-07.	El Domain Service podía invalidar una elegibilidad ya obtenida; un mismo Value Object mezclaba cancelación del Evento y cierre de solicitudes.	Se conservaron las siete raíces y los dos Domain Services. La separación de motivos y los temas de negocio no resueltos quedaron pendientes; no se afirmó haber revisado un BPMN que no estaba disponible.
4	Etapa 4: Revisar la Service Layer y la coordinación mediante Domain Events.	(Christopher): «Audita los Application Services para que coordinen los casos de uso sin acumular reglas de dominio. Revisa los métodos de anticipo, regularización y contratación, anexos y confirmaciones; evita notificaciones duplicadas y responde las preguntas de revisión de la guía con evidencia».	Se identificaron métodos que modificaban más de una raíz, una notificación duplicada y responsabilidades repetidas para SE-012.	Había métodos que modificaban más de una raíz, notificaciones duplicadas y responsabilidades repetidas para SE-012.	Se corrigió el proceso: el vencimiento de la regularización genera un aviso, pero no cancela el Evento por sí solo. El Coordinador debe registrar la cancelación. Las demás correcciones se comprueban contra el código y las pruebas.
4.1	Etapa 4: Corregir el proceso de cancelación por saldo vencido.	(Christopher): «Conforme a BN-005, BN-008 y CA-027, si vencen las 24 horas sin pago registrado y verificado, Administración informa al Coordinador. El Coordinador registra la cancelación, su motivo y los avisos correspondientes. Distingue este proceso del cierre por anticipo no recibido y actualiza los componentes afectados».	Se separó el aviso de vencimiento de la acción de registrar la cancelación; se detectó también un defecto en el orden de verificación del pago.	La propuesta inicial trataba la cancelación como automática y se detectó un pago marcado como liquidado antes de la verificación administrativa.	Se acordó que no haya cancelación automática por saldo vencido. Sigue pendiente definir el plazo para que el Coordinador registre la cancelación (P-08). El pago solo cuenta como liquidado cuando se verifica.
5	Etapa 5: Definir Repository, Unit of Work y límites transaccionales.	(Christopher): «Define interfaces Repository por raíz de agregado, separadas de la infraestructura. Completa la tabla de Unit of Work con inicio, cambios confirmados juntos y manejo de errores; incluye el cierre por anticipo no recibido y explica cómo reducir el riesgo de perder Domain Events tras el commit».	Se organizaron interfaces por agregado y operaciones transaccionales; se propuso Outbox como alternativa para manejar eventos después del commit.	Una reacción confirmaba dos agregados en una sola transacción y la tabla de Unit of Work tenía una fila duplicada.	Se mantuvieron los repositorios por raíz y los límites transaccionales propuestos. Outbox sigue como propuesta en ADR-004; no se presenta como almacenamiento durable porque la implementación es en memoria.
6	Etapa 6: Aplicar CQRS básico separando comandos, consultas y modelos de lectura.	(Christopher): «Aplica CQRS sin bases de datos separadas. Separa Commands y Queries, evita duplicar datos financieros en SolicitudPendienteViewModel y haz que ConsultarEstadoPagosQuery reciba eventoId y consulte las cuentas correspondientes».	Se separaron Commands, Queries y Handlers y se ajustaron los View Models para evitar duplicar información financiera.	SolicitudPendienteViewModel incluía datos financieros que correspondían al estado de pagos.	Se adoptó CQRS básico sin separar bases de datos. Los comandos modifican el estado y las consultas recuperan información; los modelos de lectura deben evitar datos financieros duplicados.
7	Etapa 7: Evaluar SOLID y construir la matriz de trazabilidad y el código base.	(Francisco): «Revisa las etapas 3 a 6 con los cinco principios SOLID. Registra hallazgo, evidencia, impacto y corrección o justificación. Construye la matriz BN, SE, US, CA y CU sin inventar relaciones y organiza el código en domain, application e infrastructure».	Se documentaron hallazgos H-01 a H-09 y se preparó una base Python en tres capas. Algunas relaciones de trazabilidad quedaron inferidas porque no se disponía de BPMN.	La falta de BPMN impidió confirmar algunas relaciones; ciertos vínculos SE–US/CA se infirieron por el contenido.	Se aceptó elaborar la matriz y registrar los hallazgos SOLID. Las relaciones que no se pueden confirmar, especialmente sin BPMN, deben revisarse y no marcarse como seguras sin evidencia.
7.1	Etapa 7: Corregir defectos técnicos antes de elaborar los diagramas.	(Francisco): «Verifica que un cambio confirmado se aplique al Evento, que los servicios adicionales no queden con estado ambiguo, que una sustitución tardía no reutilice una fecha vencida y que no haya avisos duplicados. Escribe pruebas para los defectos, conserva las reglas aprobadas y mantén pendientes las decisiones no definidas».	Se agregaron pruebas que detectaron defectos en el dominio, los servicios y las reacciones. Se corrigieron las inconsistencias identificadas y se unificó el modelo en v1.4.	Una prueba asumía una ruta que no correspondía a BN-003; el código tuvo que reconstruirse desde la versión entregada tras reiniciarse el entorno.	Se mantuvo P-06 como pendiente. Se corrigieron las pruebas que esperaban un comportamiento no respaldado por BN-003 y se conservaron como pendientes las reglas que la documentación no define.
8	Etapa 8: Elaborar diagramas UML y decisiones arquitectónicas.	(Francisco): «Construye un diagrama de clases coherente con el mapa, los servicios y los repositorios, separando dominio,	Se elaboraron tres vistas de clases, secuencias para CU-02 y CU-03 y ADR-001	Los ADR contenían identificadores que no correspondían a la regla; la secuencia colocaba	Se conservaron ADR-001 y ADR-003 como decisiones documentadas. ADR-002 y ADR-004 siguen siendo propuestas, por lo que no deben presentarse como aprobadas.

		aplicación e infraestructura. Añade dos secuencias editables y legibles y al menos tres ADR con contexto, alternativas, decisión, consecuencias y trazabilidad. Usa solo clases y métodos existentes».	a ADR-004. Se ajustó la rama de confirmación para que no apareciera después de un rechazo.	confirmar() después de un rechazo.	
9	Etapa 9: Validar los flujos, invariantes, transacciones y reacciones; documentar resultados.	(Francisco): «Ejecuta las pruebas de los flujos principales, alternativos y de error de CU-01 a CU-05, las invariantes y los límites transaccionales. Corrige solo defectos reales, vuelve a ejecutar y registra resultados, evidencias, observaciones y casos no validables. No afirmes pruebas que no se ejecutaron».	El registro previo reportó 70 pruebas aprobadas, defectos D-01 a D-07, observaciones O-01 y O-02 y casos no validables.	Las pruebas iniciales incluyeron errores en nombres/importaciones y un escenario no alcanzable; GitHub no se publicó.	Quedaron pendientes O-01, O-02 y los casos que necesitan una decisión del negocio. También falta publicar el repositorio en GitHub. Los resultados históricos no sustituyen una nueva ejecución de pruebas.
10	Analizar de forma integral el flujo del negocio y localizar inconsistencias entre BPMN y documentos.	(Lesly): «Analiza detalladamente el flujo expresado en la documentación. Compara el BPMN con el BRD, las historias, criterios y glosario; identifica procesos faltantes, contradicciones y puntos que necesitan corrección. Indica con precisión dónde corregir cada documento y no supongas que el BPMN fue revisado si no está disponible».	Se enfocó la revisión en procesos de cambios, servicios adicionales, cancelaciones, pagos, autorizaciones y seguimiento de proveedores, contrastando reglas y criterios entre documentos.	El historial original no conserva todos los detalles de la revisión; los hallazgos aquí se resumen sin atribuir citas literales.	Se priorizó corregir las reglas y las relaciones entre documentos. Los procesos que el stakeholder no ha definido se mantienen pendientes en lugar de completar los vacíos con suposiciones.
11	Unificar plazos, pagos y reglas de cambios y servicios adicionales.	(Lesly): «Verifica que los tres documentos distingan correctamente los días hábiles y naturales para anticipo, saldo, cambios y servicios adicionales. Comprueba también el saldo vencido, las 24 horas de regularización, los costos del proveedor, el cargo fijo de Eventos Nova y los casos de cierre y devolución».	Se consolidaron los plazos: anticipo dentro de 3 días hábiles; cambios con 7 días naturales; servicios adicionales con 5 días naturales; regularización de saldo y pago de servicio adicional en 24 horas naturales según las condiciones documentadas.	Los plazos debían distinguir días hábiles de naturales y separar los cargos de Eventos Nova de los costos del proveedor.	Se unificaron los plazos distinguiendo días hábiles y naturales, y se separaron los cargos de Eventos Nova de los costos de proveedores. Las devoluciones se revisan junto con la política de no reembolso y las excepciones aplicables.
12	Revisar la matriz y sus relaciones con las historias y criterios.	(Lesly): «Comprueba que cada necesidad y regla esté relacionada con las historias, criterios, casos de uso y componentes que realmente la implementan. Revisa especialmente los vínculos faltantes, los identificadores CA y las relaciones inferidas; señala qué debe cambiarse y dónde».	Se identificó la necesidad de revisar relaciones faltantes, evitar que rangos de criterios oculten omisiones y distinguir relaciones explícitas de las inferidas.	La matriz no debe convertir relaciones inferidas en confirmadas ni ocultar omisiones con rangos amplios de criterios.	La matriz debe mostrar relaciones comprobables, no aparentar una cobertura total. Se conservan solo los vínculos respaldados y se señalan los que siguen pendientes.
13	Revisión de consistencia: validar como stakeholder las reglas dudosas o pendientes y corregir solo los artefactos afectados.	(Equipo): «Analiza todo el proyecto antes de modificar archivos; actúa como stakeholder de Eventos Nova; decide O-01, O-02, O-03, P-04, P-06, P-08, los días hábiles y el cambio de fecha con la opción más sencilla dentro del alcance; corrige solo los artefactos afectados y conserva la trazabilidad».	Se comprobó que la consulta a proveedores forma parte de la viabilidad de todo cambio; se decidieron O-01, O-02, P-04, P-06 y P-08; se definieron los días hábiles (GL-060) y se dejó fuera del alcance reprogramar la fecha. Se encontraron dos defectos (D-08 y D-09) y una prueba documental que fallaba por el texto del reporte resumen.	El enunciado de la revisión resumía un pago final 7 días antes del evento y aplicaba al servicio adicional la regla de la sustitución; los documentos establecen 3 días hábiles y el cierre como No autorizado. P-01, P-02, P-03 y P-07 no tienen una definición en los documentos.	Se conservaron las reglas documentadas y no se inventaron los pendientes no verificables. Los cambios propuestos con la IA deben revisarse por el equipo antes de la entrega.
14	Corrección final: cerrar con el motivo «Evento cancelado» las solicitudes en trámite al cancelarse el Evento y auditar la versión corregida contra la anterior.	(Equipo): «Corrige solo lo necesario sobre la versión actual: al cancelarse el Evento, toda solicitud en trámite que no esté confirmada se cierra con el motivo "Evento cancelado"; verifica fecha y hora, CA-058, pagos, cambios, servicios adicionales, días hábiles y roles; audita contra la versión anterior y regenera los entregables».	Se agregó el estado «Cerrada por evento cancelado» en las tres solicitudes y una reacción a EventoCancelado que las cierra y aplica SE-012; se actualizaron BRD, criterios, glosario, informe, matriz, mapa de dominio y diagramas. Las demás reglas se verificaron sin cambios.	La corrección también describía que, si Administración no autoriza la alternativa de un servicio adicional, se siga buscando; esa es la regla de la sustitución de proveedor. Los documentos no definen qué ocurre con una operación ya confirmada que todavía no se aplicó al Evento cuando este se cancela.	Se conservó el cierre del servicio adicional no autorizado (CA-081). La situación de la operación confirmada pendiente de aplicar se registró como ambigüedad sin crear una regla nueva. Los cambios deben revisarse por el equipo antes de la entrega.
15	Corrección final del caso pendiente:	(Equipo): «Si una operación ya confirmada no se aplicó antes de	Evento rechaza la operación confirmada	Los documentos no definen quién atiende el	No se creó un estado nuevo de solicitud ni un proceso de devolución o de cobro; la

	operación confirmada que todavía no se aplicó cuando se cancela el Evento; auditoría completa y BPMN con fuente editable.	cancelarse el Evento, no la ejecutes sobre el Evento cancelado, conserva la confirmación y el historial y regístrala para revisión con el mecanismo existente; distingue la solicitud en trámite, la operación confirmada pendiente y el Evento activo (O-02); audita todos los artefactos y elabora un BPMN 2.0 completo con fuente editable».	cuando ya está Cancelado y el motivo queda en el registro del Outbox, que pasa a «Revisión»; la solicitud sigue Confirmada. Se actualizaron BRD, criterios, glosario, informe, ADR, matriz, mapa de dominio y pruebas (82). Se elaboró un BPMN general y ocho subprocesos en BPMN 2.0, validados al importarlos con bpmn-js.	caso registrado para revisión ni en qué plazo. El BPMN anterior no tenía fuente editable y no mostraba el cierre de las solicitudes al cancelar el Evento.	ambigüedad residual se reportó. El BPMN solo incluye actividades respaldadas por los documentos. Los cambios deben revisarse por el equipo antes de la entrega.
16	Auditoría final y reconstrucción del BPMN: maestro completamente desplegado, comprobación del estado del Evento en SP-05 y correcciones documentales.	(Equipo): «Usa la versión final como línea base; construye un BPMN maestro sin ocultar los procesos detrás de actividades colapsadas, con colores por rol, leyenda y una versión paginada; agrega en SP-05 la comprobación del estado del Evento al recibir el aviso del proveedor sin inventar resultados; corrige la matriz, el texto del ADR-001 y las capturas de las pruebas, y valida todo sin modificar código ajeno al objetivo».	Se generó el BPMN maestro con SP-01, SP-02, SP-06 y SP-07 como subprocesos expandidos y SP-03, SP-04 y SP-05 como procesos independientes con SE-012 (SP-08) desplegado; fuente editable, PDF vectorial, SVG, PNG y versión paginada A3. Se validaron el XML, la importación en bpmn-js sin advertencias, las salidas de las compuertas y la legibilidad de las etiquetas; se completó la fila BN-006 de la matriz con los servicios adicionales y se actualizaron las evidencias de las 82 pruebas.	Los documentos no definen el resultado de un aviso del proveedor recibido después del inicio o de la cancelación del Evento, y el prototipo registra el aviso sin comprobar ese estado. Siguen sin definirse quién atiende «Revisión» y en qué plazo, las consecuencias económicas frente a los proveedores y el periodo de seguimiento del anticipo.	La comprobación se representó en el BPMN como decisión pendiente, sin inventar estados, cobros ni devoluciones y sin modificar el código. Se conservaron las reglas y los identificadores existentes. Los cambios deben revisarse por el equipo antes de la entrega.
17	Corrección final controlada: distinguir el pago efectuado por el Cliente de su registro y verificación por Administración en el anticipo, los cambios, los servicios adicionales y la regularización, y aplicar las decisiones del equipo sobre los puntos abiertos.	(Equipo): «Corrige únicamente lo necesario: el anticipo tiene 3 días hábiles y, tras el contacto, 24 horas naturales para que el Cliente pague; esas 24 horas, como las de cambios, servicios adicionales y regularización, son para pagar, no para que Administración registre o verifique. El aviso del proveedor después del inicio queda fuera del alcance, «Revisión» es el mecanismo técnico del Outbox y la contabilidad de los proveedores no se agrega».	Pago conserva la fecha en que el Cliente pagó y separa las fechas de registro y verificación; verificarPagoRegistrado() permite verificar después; el cierre por falta de pago, el vencimiento del saldo original y el resultado de la regularización no proceden mientras haya un pago efectuado a tiempo pendiente de verificación, y el cierre por anticipo exige 24 horas naturales desde el contacto. Se agregaron 9 pruebas (91 en total) y se corrigieron BRD, criterios, glosario, matriz, mapa de dominio, diagramas UML y BPMN (SP-02 a SP-06 y maestro).	Los documentos trataban las 24 horas como plazo para registrar y verificar el pago, y el prototipo podía cerrar una solicitud por la demora de Administración. No se define qué ocurre si Administración no logra verificar un pago registrado: el pago permanece registrado y la solicitud no se confirma ni se cierra por falta de pago.	Se aplicaron las decisiones del equipo sin nuevos estados, módulos ni integraciones; el aviso posterior al inicio quedó como fuera del alcance en SP-05 y se cerraron los puntos de la iteración 16 sobre «Revisión», los proveedores y el seguimiento del anticipo. Los cambios deben revisarse por el equipo antes de la entrega.
18	Auditoría de consistencia documental: comprobar los documentos contra las reglas definitivas, corregir los restos que trataban las 24 horas como plazo de registro o verificación y dejar la fuente editable del BPMN en draw.io.	(Equipo): «Deja correcta y consistente toda la documentación sin inventar reglas ni ampliar el alcance: las 24 horas son para que el Cliente pague, no para que Administración registre o verifique; separa el cargo de Eventos Nova del costo del proveedor y el servicio adicional de la sustitución; revisa GL-015, CA-037, CA-058 y la matriz; el BPMN debe tener su fuente editable en draw.io».	Se corrigieron GL-049 en el glosario, cuatro definiciones y una evidencia del informe, la regla del saldo del resumen y el motivo de cierre por falta de pago del prototipo, que decía «Pago no verificado». Se generaron las fuentes .drawio del maestro y de SP-01 a SP-08 con las mismas posiciones, textos y conexiones que	GL-015, CA-037, US-018, la separación entre cargo de Eventos Nova y costo del proveedor y el pago final de 3 días hábiles ya eran correctos. La matriz no tiene columnas de BPMN ni de pruebas: la relación con el BPMN está en el alcance de cada subproceso y la de las pruebas, en sus comentarios.	Se conservaron el .bpmn como versión BPMN 2.0 y las exportaciones como evidencia; no se agregaron reglas, estados, procesos ni funcionalidades. Los cambios deben revisarse por el equipo antes de la entrega.

			el .bpmn (1 337 elementos comparados, 0 diferencias). La suite se volvió a ejecutar: 91 pruebas, OK.		

11.2 Qué corregimos o rechazamos y por qué

Los cuatro casos más importantes fueron estos. Primero, rechazamos la cancelación automática por saldo vencido: la propuesta de la IA contradecía la regla de negocio (Administración verifica y el Coordinador registra la cancelación con su motivo), así que la cambiamos y la documentamos en ADR-003. Segundo, no aceptamos que un pago contara como liquidado antes de la verificación administrativa; ahora solo cuenta cuando se verifica. Tercero, corregimos las reacciones que confirmaban dos agregados en una sola transacción, porque rompían los límites de agregado, y dejamos el Outbox solo como propuesta, sin presentarlo como almacenamiento durable. Cuarto, ajustamos los ADR y una secuencia que tenía identificadores de regla equivocados y colocaba la confirmación después de un rechazo. En todos los casos fue importante regresar a los documentos para decidir, porque la IA no siempre distinguía lo que estaba definido de lo que era un supuesto.

12. Conclusiones

Conclusión general del equipo

Con esta actividad comprendimos mejor cómo pasar del análisis de negocio a una arquitectura de software sin perder las reglas que habíamos identificado en la Unidad 2. El BRD y las User Stories no quedaron solo como documentos de requisitos: los utilizamos para definir los casos de uso, los conceptos del dominio, los límites de las transacciones, las consultas, los diagramas y los escenarios de prueba. La elección de Domain Model fue coherente con las condiciones cruzadas de contratos, pagos, solicitudes y proveedores, aunque reconocemos que exige más trabajo de diseño y mantenimiento.

También comprobamos que una propuesta arquitectónica debe ser explicable y revisable. Las pruebas permitieron detectar o comprobar comportamientos concretos, y la revisión con SOLID ayudó a separar responsabilidades y a corregir casos como la cancelación automática que no correspondía a la regla documentada. Sin embargo, no confundimos una suite en verde con un sistema completo: falta persistencia durable e integración real con servicios externos y quedan puntos del negocio que no pudieron verificarse en los documentos. Los BPMN representan los procesos descritos en los documentos, sin convertir los asuntos no definidos en reglas aprobadas. Registrar estas limitaciones forma parte de justificar las decisiones y de evitar que la arquitectura prometa más de lo que la implementación demuestra.

Conclusión personal — Lesly Adamaris Cotlame Nieto, matrícula 2226958

Yo comprendí que el análisis de la Unidad 2 sirve para mucho más que describir un problema: puede orientar las clases, las reglas y las decisiones de la arquitectura. Aprendí a revisar las respuestas de la IA con el BRD, las historias y el glosario, en vez de asumir que una propuesta ya era correcta. Para mí, justificar las decisiones es importante porque permite explicar por qué una regla pertenece a un concepto del dominio y reconocer con honestidad lo que todavía no está definido.

Conclusión personal — Christopher Bethancourt Contreras, matrícula 2226948

Yo aprendí que los servicios de aplicación deben coordinar un proceso, pero no convertirse en el lugar donde terminan todas las reglas. La revisión del anticipo, del saldo y de la cancelación me ayudó a entender por qué el orden de las acciones y los límites de las transacciones cambian el resultado. Considero importante

documentar la decisión y sus consecuencias para evitar que, al programar, el sistema actúe de una forma distinta a la que estableció el negocio.

Conclusión personal — Francisco Mora Ruiz, matrícula 2226912

Yo comprendí que los diagramas, el código y las pruebas deben representar la misma arquitectura. Aprendí que las pruebas sirven para comprobar escenarios concretos y encontrar inconsistencias, pero no permiten afirmar que todo el sistema está terminado. Justificar las decisiones arquitectónicas y dejar las limitaciones por escrito ayuda a que otras personas puedan revisar el trabajo, repetir las pruebas y saber qué se tiene que completar después.

Link de Repositorio de GitHub:

https://github.com/leslynieto64-del/eventos_nova_equipo16_CLNEG_004